1. I would implement the `/co-area-drivers` query in SQL by relying on self-joins on a Trips table. Specifically, the query structure will be:
    a. First, we identify all rows in the Trips table belonging to the target *driver_id* to get the list of *area_ids*.
    b. We then join that result back onto the Trips table on the *area_id* column to find every other trip records with those same areas.
    c. Filter out the original *driver_id* from the results, group the remaining rows by other *driver_ids*, and perform a *COUNT(DISTINCT area_id).*
    d. Finally, *ORDER BY* on the result of that count.
2. In a native graph database like Neo4j, this traversal is more natural because of Index-Free Adjacency (IFA). In a graph, each node, representing a driver or area, physically stores the memory addresses (pointers) of its neighboring nodes (*adjacency*). To find co-drivers, the engine simply follows a direct pointer from the area node to the connected Driver nodes. On the otherhand, SQL follows relationships. This usually involves performing an index lookup.
3. An example of a workload that Neo4j might underperform is a workload that requires massive scans of non-relational attributes: Calculate the average fare for all trips in the last year.

    Assuming that we have used the “Trip Start Timestamp” and “Trip End Timestamp” to process and record only the trips of last year.

    - To answer this, Neo4j has to visit every "Trip" node, load the entire node into memory, and extract the fare property. Because nodes are stored as discrete objects in a web of pointers, this involves a lot of arbitrary jumps in memory (random I/O).
    - A relational database like SQL stores all fare values for every trip contiguously on disk. It can stream that single column into the CPU extremely fast, performing average in a fraction of the time that Neo4j takes to jump around in memory.